

Extra Project

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Projects

This project is strictly individual. Select **one** of the following projects.

The repository must contain all relevant scripts, configuration files, and a `README.md` with instructions on how to deploy the project

This is the **extra project**, designed to integrate the various skills acquired throughout the semester (Shell Scripting, Docker, Python, Data Analysis, and Web Technologies).

This is a three-week project (deadline **07/07/2026**). You have until the end of this week to notify your professor (via e-mail) of your chosen topic (the list of topics can be found [here](#)).

Do not forget to contact your professor with any questions. Further instructions may be added.

Topics

1. Smart City Traffic & Fleet Monitoring Platform

- **Description:** Deploy a smart city transit monitoring pipeline. Multiple vehicle simulator containers (Python scripts) publish GPS coordinates, speed, and fuel telemetry via **MQTT**. A **Mosquitto** broker routes this telemetry. A **FastAPI** collector service subscribes to the MQTT topics, validates the incoming logs, and stores them in a **PostgreSQL** database. **Grafana** is deployed to visualize vehicle routes and average speed trends in real-time. A **Caddy** (or Nginx) container serves as a caching reverse proxy in front of the Grafana dashboard. A **Bash backup script** runs periodically via `cron` inside a dedicated backup container to export the PostgreSQL database, compress it into a `.tar.gz` file with a timestamp, and rotate old archives (retaining only the 7 most recent).

- **Core Skills:** Docker Compose (multi-container bridge network), MQTT (Mosquitto), SQL Persistence (PostgreSQL with volume mounts), Reverse Proxy & Caching (Caddy/Nginx), Bash Scripting & Automation (cron, tar, file rotation), Grafana.

2. Collaborative Shared Document Workspace with Version Analytics

- **Description:** Create a shared collaborative real-time writing pad. The web application allows multiple users to write Markdown, synchronizing text changes across clients instantly via **WebSockets**. The backend uses **Redis Pub/Sub** to manage character broadcast concurrency and stores all document version histories in a **PostgreSQL** database. A separate, custom offline container runs a **Python script** using **Pandas** or **Polars** to read the database periodically, compute key metrics (such as word counts, character additions, and contribution percentages per student/author), and save a markdown analytical summary report. A **Bash daemon script** runs on the host (or a privileged container) to monitor the storage directory sizes, log database container resource usage, and append heartbeats to a system log. Developers must use a **Dev Container** setup (Dockerfile configuring debugging tools and linters) for local development.
- **Core Skills:** WebSockets, Redis Pub/Sub, PostgreSQL, Dev Containers (Dockerfile), Python Data Analytics (Pandas/Polars), Bash Daemon Scripting (system health and disk monitoring), Docker Compose.

3. Global Air Pollution & Public Health Ingestion Engine

- **Description:** Build a geographical dashboard that visualizes global air pollution metrics. A **Python service** periodically queries an **external REST API** (e.g., OpenAQ API) for air pollutant densities (PM2.5, NO2, SO2) in 5 pre-selected international capitals, persisting raw data in a database. A separate analytical **Python script** using **Pandas** runs to detect safety threshold breaches, compute standard deviations (volatility), and export structured JSON data to a shared volume. An **Nginx** web server hosts a static web portal containing an **interactive Leaflet.js map** that reads the JSON data and displays color-coded markers based on air quality severity layers. A **Bash script** runs on a timer to check the external API's availability (using curl), inspect the REST rate-limits, and append status logs.
- **Core Skills:** External REST APIs, Data Analysis (Pandas), Map Visualization (Leaflet.js), Nginx Web Server, Bash Automation (curl API checking), Docker Compose.

4. Secure Industrial IoT Telemetry Vault & Decryptor

- **Description:** Implement a secure industrial log vault that automates metadata collection and cryptographic protection. A **FastAPI** web service handles authenticated device log uploads (CSV or raw files) and automatically scrapes machine metadata based on the header identifiers, storing metadata in a database. When a new log file is uploaded, a background **Bash script** acting as a directory watcher detects the file, verifies its integrity (SHA256 hashing), encrypts it using **GnuPG** or **OpenSSL** with a secure machine passphrase/key, and deletes the unencrypted original. The system must enforce strict **Linux file permissions** (chmod 600/chown in the volume) so that only the encryption agent can read raw files. A web frontend displays the machine metadata catalog and allows authorized operators to request, decrypt, and inspect machine logs on the fly.
- **Core Skills:** Security Concepts (GnuPG/OpenSSL encryption, SHA256 hashing), Linux System Permissions (chmod/chown), FastAPI Uploads, Database persistence, External REST API scraping, Bash Directory Watching.

5. Database-Driven Sports Analytics Report CI/CD Pipeline

- **Description:** Design an automated Continuous Integration pipeline that compiles sports analytics reports from database records. A multi-container application stores live match stats or team metrics in a **PostgreSQL** database. A **Bash script** monitors a PostgreSQL data view or log folder for updates. When a change is detected, it triggers a custom **Docker container** equipped with **Pandoc**, **LaTeX**, and **Python**. This container executes a Python script that runs queries, analyzes the sports database using **Pandas**, generates statistical plots (PNGs), updates a dynamic **Markdown** template, and compiles it via **Pandoc** into a final PDF report. The compiled PDF is automatically copied to a volume served by a **Caddy** web server, allowing coaches to download the latest athletic report.
- **Core Skills:** Custom Dockerfiles (Pandoc + LaTeX + Python), Bash Automation (database/folder monitoring), Python Plotting (Matplotlib/Seaborn), Pandoc PDF compilation, SQL Databases, Caddy/Nginx web serving.

6. Warehouse Asset Inventory System with Technical Wiki

- **Description:** Create a smart warehouse tracking pipeline supported by interactive wiki documentation. Simulators publish barcode scan signals and inventory movements via **MQTT** to a **Mosquitto** broker. A **FastAPI** collector stores this telemetry in **PostgreSQL**. **Grafana** is deployed to display inventory trends and trigger alerts when stock

levels fall below thresholds. To document the system's operational architecture, a **BookStack** or **DokuWiki** container is deployed alongside the system, pre-populated via a mounted volume with at least 5 markdown pages detailing container networking subnets, inventory database schemas, and setup instructions. A **Bash script** runs periodically to check CPU and memory usage of the containers, logging the resource footprint.

- **Core Skills:** MQTT (Mosquitto), PostgreSQL, Grafana, Wiki Deployment (BookStack/DokuWiki), Docker Volumes & Networking, Bash Resource Telemetry.